

GUIA: Configuração do Alerta TradingView — Catalyst V2 + TSTS + RSI (5m)

Visão Geral

Este guia mostra como configurar o alerta unificado que:

1. **Detecta** sinal TSTS (seu indicador BSDDET ou similar)
2. **Confirma** que o RSI cruzou a média (até 3-4 velas atrás)
3. **Calcula** SL/TP automaticamente
4. **Envia** JSON completo ao bot com dados do catalisador
5. **Bot checa** `catalyst.checar()` → se aprovado, simula entrada

PASSO 1: Adaptar a Lógica TSTS no Pine

O Pine script fornecido (`catalyst_v2_tsts_rsi_5m.pine`) usa **cruzamento de EMAs como EXEMPLO**.

IMPORTANTE: Substituir pela sua lógica TSTS real

Localizar no Pine (linhas ~95-105):

```
// SEÇÃO 3: LÓGICA DE SINAL TSTS (ADAPTE AQUI)
// EXEMPLO GENÉRICO: cruzamento de EMAs.
// SUBSTITUA pela saída do seu BSDDET Helper ou outro indicador TSTS.
ema9  = ta.ema(close, emaRapida)
ema21 = ta.ema(close, emaLenta)

sinTstsBuy  = ta.crossover(ema9, ema21)
sinTstsSell = ta.crossunder(ema9, ema21)
```

Como adaptar:

Opção A: Você já tem um indicador TSTS separado (BSDDET Helper)

Se você usa outro script Pine que gera sinais `buy / sell`, você precisa **integrá-lo** neste script. Exemplo:

```
// Importa seu indicador (se for biblioteca pública) ou copia a lógica
// Exemplo: BSDDET Helper v4 (assumindo que você tem acesso)

// Variáveis do BSDDET (adapte conforme seu indicador)
bsdetBuy  = ... // condição de compra do BSDDET
bsdetSell = ... // condição de venda do BSDDET

// Substitui as linhas acima por:
sinTstsBuy  = bsdetBuy
sinTstsSell = bsdetSell
```

Opção B: Usar indicador externo via `request.security`

Se seu BSDET está em outro gráfico/script:

```
// Exemplo: indicador "BSDET Helper v4" no mesmo ticker
// Ajuste o nome exato do seu indicador
sinTstsBuy = request.security(syminfo.tickerid, "5", bsdet_buy_signal_function())
sinTstsSell = request.security(syminfo.tickerid, "5", bsdet_sell_signal_function())
```

Opção C: Manter EMAs para teste inicial

Se quiser testar o sistema primeiro, deixe as EMAs. Elas vão gerar sinais frequentes (bom para validar o webhook).



PASSO 2: Adicionar o Indicador no TradingView

1. **Abra** o gráfico da moeda desejada (ex: `BTCUSDT`) no timeframe **5m**
2. **Pine Editor** → Cole o código de `catalyst_v2_tsts_rsi_5m.pine`
3. **"Add to Chart"**
4. **Verifique o painel** no canto superior direito:
 - Deve mostrar os TFs (30s, 1m, 5m, 15m, 1h, 2h, 4h)
 - VWAP, Market (RANGING/TRENDING), Pullback, Grade
 - RSI atual vs média
5. **Setas** verdes (▲) e vermelhas (▼) devem aparecer quando houver gatilho



PASSO 3: Criar o Alerta

3.1 Configurações Básicas

Campo	Valor
Condição	Catalyst V2 + TSTS + RSI (5m) → Any alert() function call
Opções	☒ Once Per Bar Close
Expiração	Open-ended (ou defina um prazo)
Nome do alerta	[5m] BTCUSDT Catalyst V2 (adapte para cada moeda)

3.2 Webhook URL

Para cada moeda, use a URL específica:

```
https://web-production-77454.up.railway.app/webhook/{{ticker}}
```

Exemplos:

- BTCUSDT: `https://web-production-77454.up.railway.app/webhook/BTCUSDT`
- ETHUSDT: `https://web-production-77454.up.railway.app/webhook/ETHUSDT`
- SOLUSDT: `https://web-production-77454.up.railway.app/webhook/SOLUSDT`

⚠ **IMPORTANTE:** O `{{ticker}}` será substituído automaticamente pelo TradingView.

3.3 Mensagem do Alerta

Campo “Message”:

```
{{strategy.order.alert_message}}
```

Isso faz o TradingView enviar exatamente o JSON que o Pine montou.



PASSO 4: Exemplo de JSON Enviado

Quando o alerta dispara, o webhook recebe algo assim:

```
{
  "moeda": "BTCUSDT",
  "action": "buy",
  "entry": 63250.50,
  "sl": 62934.25,
  "tp": 63882.75,
  "timeframe": "5m",
  "rsi_valor": 58.34,
  "cruzamento_numero": 2,
  "c30s": "BULL",
  "c1m": "BULL",
  "c5m": "BULL",
  "c15m": "BULL",
  "c1h": "NEUT",
  "c2h": "BULL",
  "c4h": "BULL",
  "vwap": "BULL",
  "market": "TRENDING",
  "pullback": "NONE",
  "grade": "B"
}
```

Campos Explicados

Campo	Descrição
moeda	Ticker (BTCUSDT, ETHUSDT...)
action	buy ou sell
entry	Preço de entrada (close da vela)
sl	Stop Loss (calculado em % configurado)
tp	Take Profit (calculado em % configurado)
timeframe	5m (fixo neste setup)
rsi_valor	Valor do RSI no momento do sinal
cruzamento_numero	1º, 2º, 3º... cross (quantas velas atrás)
c30s até c4h	Estado de cada timeframe (BULL/BEAR/NEUT)
vwap	Posição vs VWAP
market	RANGING ou TRENDING
pullback	BULL (recuo em alta) / BEAR / NONE
grade	A/B/C (força do contexto)

PASSO 5: Testar o Sistema

5.1 Verificar Recepção no Bot

Após criar o alerta, **dispare manualmente** (clique no ícone ► ao lado do alerta) ou espere um sinal real.

Confira no endpoint `/diag` :

```
curl https://web-production-77454.up.railway.app/diag
```

Procure por `ultimos_eventos` → deve mostrar:

```
{
  "timestamp": "2026-08-07T22:45:10Z",
  "moeda": "BTCUSDT",
  "evento": "webhook recebido",
  "payload": { ... }
}
```

5.2 Verificar Processamento do Catalisador

No log do Railway, procure por:

```
[CATALYST] BTCUSDT buy → OK (regra=R1, grade=B)
```

Ou, se bloqueado:

```
[CATALYST] BTCUSDT buy → BLOQUEADO (motivo: timing 1m contra)
```

5.3 Verificar Registro de Sombra

Endpoint /registro :

```
curl https://web-production-77454.up.railway.app/registro | tail -20
```

Deve mostrar entradas simuladas com todos os campos do JSON.



PASSO 6: Ajustar Parâmetros (Opcional)

No Pine, você pode ajustar via inputs:

Parâmetro	Padrão	Efeito
rsiCrossLookback	4 velas	Tolerância para RSI cross “an-tigo”
slPct	0.5%	Stop Loss
tpPct	1.0%	Take Profit
rangingMult	0.5	Sensibilidade do filtro RAN-GING
usarMacro	true	Enviar c2h/c4h (ligar para 5m)



PASSO 7: Replicar para Outras Moedas

Para cada moeda adicional:

1. **Abra o gráfico** da moeda (5m)
2. **Adicione o mesmo indicador** (sem modificar o código)
3. **Crie um novo alerta** com:
 - Webhook URL específico da moeda
 - Nome do alerta identificando a moeda

4. **Verifique** no `/diag` que o bot reconhece a nova moeda



PASSO 8: Acompanhar Resultados

Logs Diários

Consulte os arquivos:

- `/shared/crypto2_logs/crypto_2026-08-07.json` (máquina)
- `/shared/crypto2_logs/crypto_2026-08-07.md` (humano)

Resumo no Telegram

Todo dia às 23:59 UTC, o bot envia:

- Total de sinais recebidos
- Entradas simuladas por moeda/TF
- Win rate por grade (A/B/C)
- Win rate por regra do catalisador



Troubleshooting Comum



Alerta não dispara

- Verifique se há setas no gráfico (▲ buy, ▼ sell)
- Confirme que “Once Per Bar Close” está ativo
- Teste com lookback maior (6-8 velas) temporariamente



Webhook retorna erro 400

- JSON malformatado → confira se a mensagem é `{{strategy.order.alert_message}}`
- Campos faltando → verifique se o Pine montou o JSON completo



Bot bloqueia todas as entradas

- Verifique `config.json` → `bloquear_ranging` deve estar `false`
- Verifique `bloquear_contra_macro` → deixe `false` por ora (coleta de dados)
- Confira `/diag/catalyst/BTCUSDT` para ver os TFs em tempo real



Cruzamento RSI não detectado

- Aumente `rsiCrossLookback` para 6-8 velas
- Verifique visualmente no painel se RSI > média quando espera cross up



Checklist de Ativação

- [] Pine adaptado com lógica TSTS real (ou EMA para teste)
- [] Indicador adicionado no gráfico 5m
- [] Painel visual mostra TFs corretos
- [] Alerta criado com webhook correto
- [] Mensagem = `{{strategy.order.alert_message}}`

- [] Teste manual enviado (clique no ► do alerta)
- [] `/diag` mostra webhook recebido
- [] `/registro` mostra entrada simulada
- [] Log do Railway mostra processamento do catalisador
- [] Replicado para todas as moedas desejadas

Próximos Passos

Após rodar em sombra por alguns dias:

1. **Analisar win rate** por grade e por regra
2. **Decidir** quais regras/grades podar
3. **Calibrar TP/SL** baseado em MFE/DD real (dados do Postgres)
4. **Configurar** `bloquear_contra_macro: true` quando for pro real
5. **Testar** com alavancagem 5x (baseline) e 10x (só grade A)

Dúvidas? Verifique:

- Logs do Railway: `https://railway.app` → seu projeto → Deployments → Logs
- Endpoint de diagnóstico: `https://web-production-77454.up.railway.app/diag/catalyst/BTCUSDT`
- Arquivo de config: `/home/ubuntu/bot_tsts_sniper/config.json`